

Documento di Analisi e Progettazione

Premessa

Sono uno sviluppatore .NET con molti anni di esperienza alle spalle. Tuttavia ci tengo a precisare che questo progetto rappresenta una delle mie primissime esperienze pratiche con la realizzazione di un'architettura distribuita a microservizi.

Per ottimizzare i tempi e affrontare tecnologie per me nuove (come MongoDB, i manifest per Kubernetes e la configurazione di SonarQube), ho utilizzato l'AI come assistente. **Tuttavia, la guida del progetto e le scelte architetturali sono state interamente mie.** Inoltre, date le tempistiche della prova, ho preso alcune decisioni:

1. Come consentito dalla traccia, ho implementato solo un sottoinsieme dei casi d'uso (es. non tutti i cambi di stato possibili sono gestiti). Ho preferito concentrarmi sulla solidità dell'architettura e sulla qualità del codice piuttosto che su un completamento di tutte le CRUD.
2. In un contesto reale avrei probabilmente isolato le logiche comuni in pacchetti NuGet privati. Per questa prova, ho optato per un più rapido progetto condiviso (**BuildingBlocks**) all'interno della stessa solution.

Durante lo sviluppo, l'utilizzo di Postman mi ha fatto capire che i classici Unit Test non sarebbero stati sufficienti; questo mi ha portato a implementare test di integrazione End-to-End (E2E) reali per collaudare l'intero flusso.

1 - Visione d'insieme e Architettura

1.1 - API Gateway e Accesso Centralizzato

Il sistema ha una sola "porta d'ingresso" (API Gateway) realizzata con YARP.

- **Perché l'ho scelto:** L'idea è quella di avere un "direttore d'orchestra" che centralizza tutto il traffico esterno. Il Gateway fa da barriera di sicurezza gestendo l'autenticazione e, soprattutto, **nasconde le rotte dei microservizi sottostanti**. Chi chiama le API da fuori non sa (e non deve sapere) come è strutturato il backend internamente.
- **Responsabilità:** Riceve tutte le richieste esterne e le smista al servizio corretto.
- **Sicurezza:** Il Gateway controlla che il token JWT sia valido. Se non lo è, blocca subito la richiesta. Se è valido, legge le informazioni dell'utente e le passa ai servizi interni in modo sicuro.
- **Vantaggio:** In questo modo, il codice del *Document Service* rimane pulito perché non deve preoccuparsi di validare i token, ma si fida di ciò che gli passa il Gateway.

1.2 - Suddivisione dei Microservizi e Confini Funzionali

Il sistema è diviso in due microservizi principali per separare chiaramente le responsabilità. Per facilitare lo sviluppo sul proprio computer, l'intero progetto è gestito tramite **.NET Aspire**, che avvia tutto il necessario con un solo clic.

Identity Service

- **Responsabilità:** Gestisce le identità degli utenti e crea i token di accesso (JWT).
- **Comunicazione:** Risponde solo a chiamate dirette (sincrone) per il login e non comunica con altri servizi interni.
- **Sicurezza:** È l'unico componente che genera e firma i token.

Document Service

- **Responsabilità:** Gestisce il ciclo di vita dei documenti (Preventivi, Proforma, Ordini) e i loro passaggi di stato.
- **Comunicazione:** Usa chiamate standard (REST) per leggere e scrivere i dati. Per le modifiche, utilizza controlli per evitare doppie scritture (idempotenza). Per le letture, utilizza una memoria veloce (HybridCache).
- **Eventi asincroni:** Quando un documento cambia stato, il servizio invia un messaggio (tramite RabbitMQ e MassTransit). Un "ascoltatore" interno riceve questo messaggio e salva un log delle operazioni in un database separato per tenere traccia di tutto.
- **Sicurezza:** Tutte le operazioni richiedono un utente autenticato, ma il servizio non gestisce le password in modo diretto.

2 - Progettazione Backend .NET

2.1 - La Scelta della Vertical Slice Architecture

Invece di dividere il progetto nei classici strati orizzontali (Dati, Logica, Presentazione), il *Document Service* è stato progettato per "funzionalità" (Vertical Slice).

- **Perché l'ho scelta:** Ritengo sia l'approccio più adatto per i microservizi. Permette di rendere le singole funzionalità totalmente "atomiche" e modulari. Se in futuro devo fare una modifica capillare su come si approva un preventivo, tocco solo un file, senza rischiare di rompere altre parti del codice.

2.2 - L'uso strategico di .NET Aspire

L'intera solution è orchestrata tramite l'host di .NET Aspire.

- **Perché l'ho scelto:** Avendolo già approcciato a lavoro, ho notato i suoi enormi vantaggi per lo sviluppo locale. Mi permette di tirare su istantaneamente contenitori diversi (MongoDB, Redis, RabbitMQ) e farli comunicare tra loro **senza dover scrivere complicati file YAML di Docker**, ma usando semplice e intuitivo codice

C#. Inoltre, fornisce "gratis" una dashboard potentissima con un sistema completo di logging e tracciamento per capire sempre cosa sta succedendo.

2.3 - Struttura del Progetto

- `DocumentManagementMicroservices.AppHost`: Il "motore" di Aspire che avvia i database (MongoDB, Redis), le code (RabbitMQ) e le API.
- `DocumentManagementMicroservices.ServiceDefaults`: Progetto condiviso per la configurazione centralizzata di OpenTelemetry, Health Checks e Service Discovery.
- `DocumentManagementMicroservices.ApiGateway`: Il punto di ingresso.
- `DocumentManagementMicroservices.IdentityService`: Web API dedicata all'emissione dei token.
- `DocumentManagementMicroservices.DocumentService`: Web API contenente il core business. All'interno, il codice è raggruppato in cartelle per funzionalità.
- `DocumentManagementMicroservices.BuildingBlocks`: Una libreria condivisa che contiene codice riutilizzabile (es. gestione degli errori o sistemi di cache).

2.3 - Aspetti Trasversali

- **Caching e Idempotenza**: Per supportare l'approccio CQRS, i command handler implementano un middleware/filtro di **Idempotenza** (basato su chiavi fornite dal client nell'header HTTP) per prevenire scritture duplicate in caso di *retry*. Le logiche di lettura sono invece accelerate tramite una libreria di **Integrazione Cache** che sfrutta `HybridCache` e `Redis` (distribuita), permettendo di alleggerire il carico sul database durante la consultazione dei documenti. Per ovviare all'impossibilità di deserializzare classi astratte direttamente da Redis (dovute alla serializzazione JSON), la strategia di caching implementata prevede il salvataggio di DTO piatti e ottimizzati.
- **Gestione degli Errori e Validazione**: La validazione dell'input è gestita tramite *FluentValidation*, in modo da bloccare le richieste malformate prima che raggiungano l'handler. Le eccezioni di dominio e di sistema sono catturate da un *Global Exception Handler* che formatta le risposte secondo lo standard **Problem Details** (RFC 7807).
- **Logging e Tracing**: Configurati tramite *OpenTelemetry* e iniettati nel progetto `ServiceDefaults`. I log strutturati, le metriche e le tracce distribuite vengono raccolti ed esposti direttamente nella dashboard di .NET Aspire durante lo sviluppo locale, pronti per essere esportati verso sistemi come Prometheus/Grafana in ambiente Kubernetes.
- **Versioning API**: Implementato nativamente tramite il pacchetto `Asp.Versioning.Mvc`. Le API espongono la versione tramite URL path o header, garantendo retrocompatibilità e favorendo un'evoluzione sicura.

3 - Modellazione Dati (MongoDB)

3.1 - Modelli Principali

Trattandosi di MongoDB (database NoSQL), i dati sono salvati come documenti.

- **La divisione logica:** In un'architettura a microservizi "pura", ogni servizio (Identity, Documenti, Log) dovrebbe avere il proprio server database fisico separato. Per ottimizzare le risorse del computer e i tempi di sviluppo, ho preferito istanziare **un singolo contenitore MongoDB**, andando però a dividere i database in modo **logico** (`identitydb`, `documentdb`, `auditlogdb`). Il principio di isolamento dei dati è comunque rispettato.
- **Documento Base:** Contiene le informazioni comuni (Numero, Data, Cliente, Stato).
- **Preventivo / Proforma / Ordine:** Ereditano dal documento base ma possono avere campi specifici. MongoDB capisce in automatico che tipo di documento sta leggendo.

I log generati dagli eventi (Audit) vengono salvati in un database separato per non appesantire quello principale.

3.2 - Relazioni tra Documenti (Referencing ed Embedding)

- **Dati incorporati (Embedding):** Le righe di un preventivo vengono salvate *dentro* il preventivo stesso, perché non hanno senso da sole.
- **Riferimenti (Referencing):** Per i clienti, si salva solo l'ID e qualche dato essenziale nel documento, senza ricopiare l'intera anagrafica.
- Se un preventivo genera una proforma, quest'ultima manterrà un riferimento (ID) al preventivo originale per tracciarne la storia.

3.3 - Concorrenza e Indici

- **Concorrenza Ottimistica:** Ogni documento ha un numero di "versione". Se due utenti provano a modificare lo stesso documento contemporaneamente, il sistema se ne accorge tramite questo numero e blocca il secondo utente per evitare di sovrascrivere dati.
- **Inizializzazione:** All'avvio, il sistema controlla se il database è vuoto. In caso affermativo, crea le tabelle, gli indici di ricerca e inserisce alcuni dati di base (come gli utenti di test) in totale autonomia.

4 - Eventi Asincroni e Messaggistica

4.1 - La scelta di MassTransit e RabbitMQ

Quando un documento cambia stato, il servizio invia un messaggio asincrono tramite RabbitMQ per far salvare un log di controllo (Audit) in un database separato, senza rallentare la risposta all'utente.

- **Perché MassTransit:** Non l'ho scelto solo per inviare semplici messaggi verso RabbitMQ. L'ho integrato perché, in un'ottica di evoluzione futura, rende il sistema già pronto per gestire **operazioni transazionali complesse tra microservizi differenti (adottando il Saga Pattern)**, garantendo la consistenza dei dati anche quando coinvolgono domini diversi.

5 - Strategia di Testing e Collaudo

5.1 - L'evoluzione dei Test (Da xUnit a Postman)

Inizialmente, la strategia prevedeva la scrittura di classici **Unit Test** per verificare le regole di business (es. impedire di creare una Proforma da un Preventivo in bozza). Tuttavia, utilizzando **Postman** per testare manualmente le API, mi sono reso conto che testare il codice isolato non bastava. Era fondamentale verificare che tutto il sistema (Database, Cache, Code di messaggi) comunicasse correttamente.

Per questo motivo, ho introdotto i **Test End-to-End (E2E)** utilizzando *Testcontainers*: durante i test automatici, il sistema avvia dei veri container Docker (MongoDB, Redis, RabbitMQ), esegue le operazioni e poi distrugge tutto. Questo garantisce che il codice funzionerà esattamente allo stesso modo in produzione.

5.2 - Giro di Collaudo (Postman)

Di seguito riporto gli screen del collaudo effettuato tramite l'API Gateway, che dimostrano il corretto funzionamento dell'intera catena (Autenticazione -> Routing -> Logica di business -> Messaggistica asincrona).

Autenticazione e Sicurezza (Identity Service)

Test A (Successo - 200 OK)



Test C (Creazione Preventivo - 201 Created)

Document Management Microservices > 1. Crea Preventivo

POST `{{gatewayUrl}}/api/v1/documents/quotes` [Send](#)

Docs Params Authorization Headers 10 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "customerId": "CUST-001",
3   "validityDays": 30
4 }
```

Body Cookies Headers 6 Test Results

JSON Preview Visualize

```
1 {
2   "id": "6a286a862234011f9e2fe13c",
3   "documentNumber": "PREV-2026-21DE"
4 }
```

201 Created · 1.44 s · 311 B

Test D (FluentValidation - 400 Bad Request)

Document Management Microservices > 1. Crea Preventivo

POST `{{gatewayUrl}}/api/v1/documents/quotes` [Send](#)

Docs Params Authorization Headers 10 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "customerId": "",
3   "validityDays": -5
4 }
```

Body Cookies Headers 8 Test Results

JSON Preview Debug with AI

```
1 {
2   "title": "Validazione fallita",
3   "status": 400,
4   "detail": "Si sono verificati uno o più errori di validazione.",
5   "instance": "/api/v1/documents/quotes",
6   "errors": {
7     "CustomerId": [
8       "Il CustomerId è obbligatorio."
9     ],
10    "ValidityDays": [
11      "I giorni di validità devono essere maggiori di zero."
12    ]
13  }
14 }
```

400 Bad Request · 119 ms · 515 B

Test E (State Machine e CQRS - 200 OK)

Document Management Microservices > 4. Cambia Stato (esecuzione di RabbitMQ)

PATCH `{{documentUri}} /api/v1/documents/{{quoteId}}/status` Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "newStatus": "Complete",
3   "expectedVersion": 1
4 }
```

Body Cookies Headers 5 Test Results 200 OK 464 ms 254 B Save Response

{ JSON Preview Visualize

```
1 {
2   "id": "6a286acc2234811f9e2fe13d",
3   "oldStatus": "Draft",
4   "newStatus": "Complete",
5   "newVersion": 2
6 }
```

Generazione e Dominio

Test F (Derivazione Documento - 201 Created)

Document Management Microservices > 5. Genera Proforma da Preventivo

POST `{{documentUri}} /api/v1/documents/{{quoteId}} /proforma` Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 6 Test Results 201 Created 85 ms 332 B Save Response

{ JSON Preview Visualize

```
1 {
2   "id": "6a286c2f2234811f9e2fe13e",
3   "documentNumber": "PROF-20268689-da87",
4   "status": "Draft"
5 }
```

Event-Driven

Test G (Audit Log su MongoDB)

Mongo Express Database: auditlogdb -> Collection: AuditLogs

Viewing Collection: AuditLogs

[New Document](#) [New Index](#)

[Simple](#) [Advanced](#)

Key Value String [Find](#)

Delete all 2 documents retrieved

_id	DocumentId	Action	Details	Timestamp
  8c66bc30-7396-4468-ba1c-2e70d4ac4845	6a255ddce0110b9b9e29d25d	StatusChange	Stato modificato da Draft a Complete	Sun Jun 07 2026 12:10:37 GMT+0000 (Coordinated Universal Time)
  6a3a0cdf-3185-4f0d-9d4d-f48b4f4dbb3d	6a286acc2234011f9e2fe13d	StatusChange	Stato modificato da Draft a Complete	Tue Jun 09 2026 19:38:50 GMT+0000 (Coordinated Universal Time)

Rename Collection

auditlogdb AuditLogs [Rename](#)

6 - Infrastruttura e Deploy su Kubernetes

6.1 - Da Aspire ad Aspire

L'applicazione è progettata per poter essere ospitata su un cluster **Kubernetes**. Invece di scrivere e mantenere manualmente i file YAML (Deployment, Service, Ingress), il progetto sfrutta uno strumento chiamato **Aspire (Aspir8)**.

Aspire analizza la configurazione di *.NET Aspire* utilizzata per lo sviluppo locale e **genera automaticamente i manifest di Kubernetes**. Questo approccio:

- Azzera gli errori nella stesura dei file YAML.
- Garantisce che l'ambiente di produzione sia l'esatta replica di ciò che è stato configurato e testato sul proprio computer.
- Genera automaticamente i *ConfigMap* e i *Secret* per gestire in sicurezza le stringhe di connessione.

6.2 - Struttura e Ruolo dei Componenti Kubernetes

I manifest YAML generati (forniti nella cartella dedicata del repository) definiscono in modo isolato e coerente l'intera architettura. La struttura non utilizza file YAML monolitici, ma sfrutta un approccio gerarchico basato su **Kustomize**:

- **Orchestrazione Kustomize:** Un file `kustomization.yaml` root coordina il deploy, richiamando le singole cartelle dei componenti (`apigateway`, `documentservice`, `mongodb`, ecc.) e risorse globali come `dashboard.yaml`.

- **Deployment Stateless:** I microservizi custom e l'API Gateway sono modellati come `Deployment` (es. `apigateway/deployment.yaml`). La strategia di aggiornamento impostata è di tipo `Recreate`.
- **StatefulSet:** Le dipendenze infrastrutturali (MongoDB, RabbitMQ, Redis) sono modellate per gestire la persistenza del dato tramite volumi dedicati (`volumeClaimTemplates`), garantendo che lo stato sopravviva al riavvio dei Pod.
- **Service e Networking:** L'esposizione e la comunicazione interna avvengono tramite risorse `Service` di tipo `ClusterIP` (es. porta 8080 per il traffico HTTP interno). Il Gateway funge da unico punto di ingresso, instradando il traffico verso le repliche corrette dei microservizi.

6.3 - Gestione Configurazioni e Segreti

La separazione tra codice e configurazione sfrutta le primitive di Kustomize e Kubernetes:

- **ConfigMap via Generator:** Le impostazioni di ambiente (URL del Service Discovery, endpoint OTLP per la telemetria) non sono scritte a mano, ma generate dinamicamente all'interno dei file `kustomization.yaml` dei singoli servizi tramite la direttiva `configMapGenerator` (es. `apigateway-env`). Queste vengono poi iniettate nei container tramite `envFrom`.
- **Traduzione del Service Discovery:** Il sistema di Service Discovery locale di .NET Aspire viene tradotto in variabili d'ambiente statiche che sfruttano il DNS interno di Kubernetes (ad esempio, la rotta verso l'Identity Service diventa <http://identityservice:8080>).
- **Segreti:** Le chiavi JWT e le password dei database vengono iniettate in modo sicuro tramite risorse `Secret` (Generate da Aspire in fase di deploy tramite `secretGenerator`), evitando qualsiasi esposizione in chiaro nel repository Git.

6.4 - Scalabilità e Gestione delle Dipendenze

- **Scaling Orizzontale:** I manifest di `Deployment` generati definiscono un valore di partenza `replicas: 1`. Essendo applicazioni stateless, `identityservice` e `documentservice` sono strutturalmente pronti per scalare modificando questo parametro o implementando un *Horizontal Pod Autoscaler (HPA)* basato sul consumo di CPU/Memoria.
- **Telemetria Centralizzata:** Il deploy include il Pod `aspire-dashboard`, che raccoglie in tempo reale tramite protocollo OTLP (sulla porta 18889) log, metriche e tracce distribuite da tutti i servizi, fornendo un'osservabilità completa del cluster.

6.5 - Approccio a CI/CD

L'organizzazione dei manifest in cartelle modulari gestite da Kustomize si sposa perfettamente con metodologie GitOps. Un ipotetico flusso automatizzato prevederebbe:

1. **Build & Test:** Ad ogni push sul branch principale, esecuzione degli unit/integration test.

2. **Quality Gate:** Analisi del codice tramite l'istanza SonarQube prevista nell'architettura.
3. **Containerization:** Compilazione delle immagini tramite il target `PublishContainer` nativo del framework .NET 10 e push su un Container Registry.
4. **Deploy:** Un tool di Continuous Deployment rileva le modifiche ai file YAML nel repository o la presenza di nuovi tag immagine, applicando le differenze al cluster target tramite `kubectl apply -k`.